

SIMATS ENGINEERING
DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

COURSE NAME: Computer Vision with OpenCV COURSE CODE: DSA02

COURSE OUTCOME COVERED: CO5: Apply computer vision techniques to real-world applications such as face recognition, tracking, medical imaging, and autonomous systems. (BL3)

ASSESSMENT TOOL 2 | Weightage: 30% | Total Marks: 30

Annexure A - Pseudocode Writing Task (Algorithm Design)

Code of Conduct:

I **Gadwal Mohammad Muzammil - 192424279** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **G. M. Muzammil**

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation	
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach	
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs	
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear	
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient	

Scenario 1: Multi-Face Detection & Recognition using Eigenfaces

SCENARIO / PROBLEM STATEMENT: Algorithm Design for Real-Time Multi-Face Recognition

Task: (A) Input: A grayscale image (size $M \times N$). (B) Output: Bounding boxes of detected faces and corresponding recognized names. (C) Requirements: Handle multiple faces in the same image and varying lighting conditions.

Complete Modular Algorithm & Line-by-Line Pseudocode

ALGORITHM: MultiFace_Detection_And_Eigenfaces_Recognition

INPUT: Grayscale Image I of size M x N, Face Database DB = {(ID_k, Name_k, Omega_k)}, Projection Matrix U_K (size D x K), Mean Face Psi (size D x 1)

OUTPUT: Annotated Image I_out with Bounding Boxes and Identity Text Labels

CONSTANTS:

CANONICAL_SIZE = (64, 64) // Feature vector dimension D = 4096
THETA_KNOWN = 2400.0 // Maximum distance threshold for verified identity
THETA_FACE = 4500.0 // Maximum subspace reconstruction error threshold

FUNCTION Preprocess_Image(image_raw):

1. Smooth sensor noise using Gaussian filter: I_smooth = GaussianBlur(image_raw, kernel=(3,3), sigma=1.0)
2. Equalize local illumination using CLAHE: I_clahe = CLAHE(I_smooth, clipLimit=2.0, tileGrid=(8,8))
3. RETURN I_clahe

FUNCTION Detect_Faces_Viola_Jones(I_proc):

1. Initialize Haar-like cascade face classifier: cascade = LoadCascade('haarcascade_frontalface.xml')
2. Multi-scale sliding window search with integral image acceleration:
face_rects = cascade.detectMultiScale(I_proc, scaleFactor=1.1, minNeighbors=5, minSize=(40,40))
3. Apply Non-Maximum Suppression (NMS) to merge overlapping candidate boxes
4. RETURN face_rects // List of bounding boxes [x, y, w, h]

FUNCTION Recognize_Face(face_crop, DB, U_K, Psi):

1. Resize cropped face to canonical size: Gamma = Resize(face_crop, CANONICAL_SIZE)
2. Normalize intensity to zero-mean and unit-variance: Gamma_norm = (Gamma - mean(Gamma)) / std(Gamma)
3. Flatten 2D matrix into 1D column vector: Gamma_vec = Flatten(Gamma_norm) // Vector in $R^{\{4096 \times 1\}}$
4. Subtract global training mean face: Phi = Gamma_vec - Psi
5. Project face onto K principal Eigenfaces subspace: Omega = U_K^T * Phi // Vector in $R^{\{K \times 1\}}$
6. Compute Subspace Reconstruction Error to verify if input is a valid face:
Phi_reconstructed = U_K * Omega
e_face = Norm2(Phi - Phi_reconstructed)
IF e_face > THETA_FACE THEN RETURN ('Invalid_Face', e_face)
7. Classify identity using Nearest Neighbor Euclidean Distance:
min_dist = INFINITY, matched_identity = 'Unknown'
FOR EACH (ID_k, Name_k, Omega_k) IN DB DO:
dist = Norm2(Omega - Omega_k)
IF dist < min_dist THEN:
min_dist = dist
matched_identity = Name_k
8. Verify recognition confidence against threshold:
IF min_dist <= THETA_KNOWN THEN RETURN (matched_identity, min_dist)
ELSE RETURN ('Unknown Person', min_dist)

MAIN PROCEDURE:

I_proc = Preprocess_Image(I)

```

face_boxes = Detect_Faces_Viola_Jones(I_proc)
I_out = Copy(I)
FOR EACH [x, y, w, h] IN face_boxes DO:
    face_crop = I[y : y + h, x : x + w]
    (label, distance) = Recognize_Face(face_crop, DB, U_K, Psi)
    Draw_Rectangle(I_out, (x, y), (x + w, y + h), color=(0, 255, 0), thickness=2)
    Draw_Text(I_out, label, (x, y - 10), font=FONT_HERSHEY_SIMPLEX, scale=0.7, color=(0, 255, 0))
RETURN I_out
END ALGORITHM

```

Scenario 2: Road Lane Detection in Real-Time Driving Video

SCENARIO / PROBLEM STATEMENT: Autonomous Vehicle Video-Based Lane Tracking Algorithm Design

Task: (A) Input: Video frames from a front-facing camera. (B) Output: Lane lines highlighted on each frame. (C) Requirements: Robust to shadows, curves, and partial occlusion.

Real-Time Autonomous Driving Road Lane Detection Pipeline

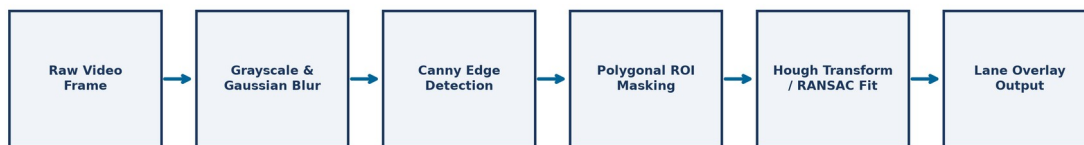


Figure 2: Real-Time Road Lane Detection Algorithm Pipeline.

Complete Algorithm Design & Pseudocode

ALGORITHM: RealTime_Autonomous_Lane_Tracking
INPUT: Video Stream from Front-Facing Camera (30 FPS)
OUTPUT: Real-Time Display with Highlighted Lane Poly-Fill Overlay

GLOBAL STATE: Left_Lane_History = [], Right_Lane_History = [], SMOOTHING_FACTOR = 0.85

FUNCTION Process_Single_Frame(Frame):

1. Illumination & Shadow-Invariant Color Masking:

```

HLS = ConvertColor(Frame, BGR2HLS)
White_Mask = InRange(HLS, lower=(0, 200, 0), upper=(255, 255, 255))
Yellow_Mask = InRange(HLS, lower=(15, 30, 115), upper=(35, 204, 255))
Color_Mask = Bitwise_OR(White_Mask, Yellow_Mask)

```
2. Edge Detection:

```

Gray = ConvertColor(Frame, BGR2GRAY)
Blur = GaussianBlur(Gray, kernel=(5, 5), sigma=1.5)
Edges = Canny(Blur, threshold1=50, threshold2=150)
Combined_Edges = Bitwise_AND(Edges, Color_Mask)

```

```

3. Region of Interest (ROI) Masking:
    Height, Width = Frame.shape[:2]
    ROI_Vertices = [(0.1*Width, Height), (0.45*Width, 0.6*Height), (0.55*Width, 0.6*Height), (0.9*Width,
Height)]
    ROI_Mask = Create_Polygon_Mask(ROI_Vertices, (Height, Width))
    Masked_Edges = Bitwise_AND(Combined_Edges, ROI_Mask)
4. Probabilistic Hough Line Extraction:
    Line_Segments = HoughLinesP(Masked_Edges, rho=1, theta=pi/180, threshold=30, minLineLength=40,
maxLineGap=25)
5. Slope Clustering & 2nd-Degree Polynomial Fitting:
    Left_Pts = [], Right_Pts = []
    FOR EACH [x1, y1, x2, y2] IN Line_Segments DO:
        slope = (y2 - y1) / (x2 - x1 + 1e-6)
        IF slope < -0.5 AND x1 < Width/2 AND x2 < Width/2 THEN Append Left_Pts, [(x1,y1), (x2,y2)]
        ELSE IF slope > +0.5 AND x1 > Width/2 AND x2 > Width/2 THEN Append Right_Pts, [(x1,y1),
(x2,y2)]
    Fit 2nd-order polynomial  $x = A*y^2 + B*y + C$  using RANSAC to eliminate outlier points.
6. Temporal Moving Average Smoothing (Handles Momentary Occlusions):
    IF Left_Pts NOT EMPTY THEN Left_Poly = SMOOTHING_FACTOR * Left_Lane_Prev + (1 -
SMOOTHING_FACTOR) * Left_Fit
    ELSE Left_Poly = Left_Lane_Prev // Hold previous track during momentary vehicle occlusion
    IF Right_Pts NOT EMPTY THEN Right_Poly = SMOOTHING_FACTOR * Right_Lane_Prev + (1 -
SMOOTHING_FACTOR) * Right_Fit
    ELSE Right_Poly = Right_Lane_Prev
7. Visual Polyfill Overlay Rendering:
    Overlay = Create_Transparent_Layer(Frame)
    Draw_PolyFill(Overlay, Left_Poly, Right_Poly, color=(0, 255, 0, alpha=0.35))
    Draw_PolyLine(Overlay, Left_Poly, color=(255, 0, 0), thickness=5)
    Draw_PolyLine(Overlay, Right_Poly, color=(0, 0, 255), thickness=5)
    RETURN AlphaBlend(Frame, Overlay)

```

MAIN VIDEO PROCESSING LOOP:

```

Capture = VideoCapture('driving_feed.mp4')
WHILE Capture.isOpened() DO:
    success, frame = Capture.read()
    IF NOT success THEN BREAK
    annotated_frame = Process_Single_Frame(frame)
    Display(annotated_frame)
Capture.release()
END ALGORITHM

```